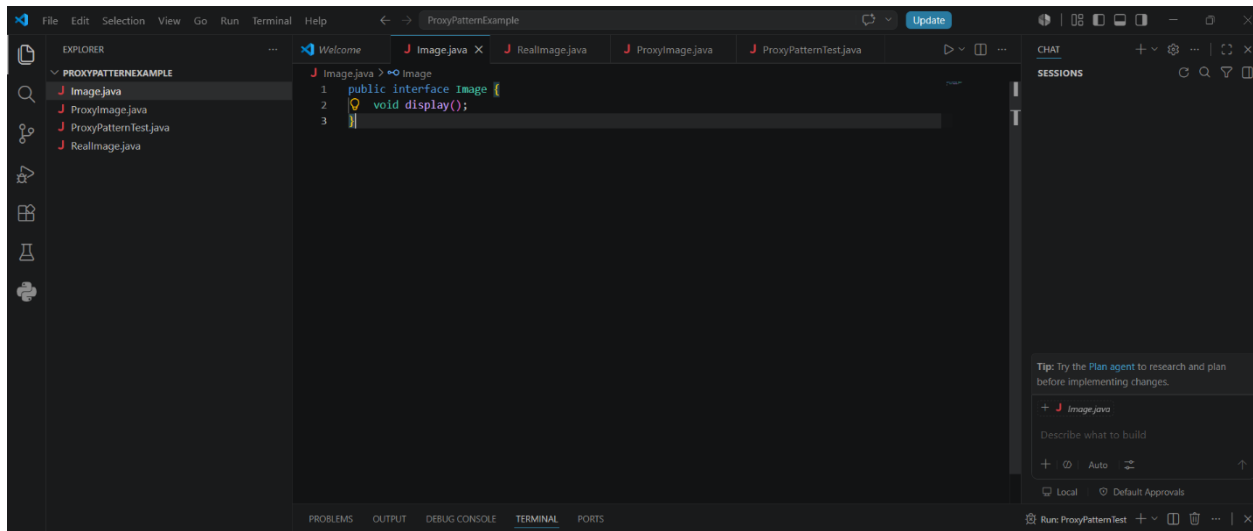


Hands-On

Exercise 6: Proxy Pattern Example

In this module, I have successfully implemented, executed and tested the Proxy Pattern in java and created a project called ProxyPatternExample. I developed an Image interface and a RealImage class that represented images retrieved from a remote server and had a display() method. I created a ProxyImage class that would implement the Image interface and would hold a reference to the RealImage. To optimize performance, I created a ProxyImage class that would implement the Image interface and would hold a reference to the RealImage. The goal of the ProxyImage class was to implement lazy initialization and caching, so that images would only be loaded when needed and reused if they were accessed more than once. Once it was implemented, I ran the program and tested the loading and display of the image using the proxy to ensure that the lazy loading and caching mechanism was working as intended. This hands-on exercise was useful to me to understand the benefits of the Proxy Pattern in terms of application performance, unnecessary resource consumption, and controlled access to resource-intensive objects.

Coding:



The screenshot shows the Visual Studio Code editor with the 'ProxyPatternExample' project open. The Explorer sidebar on the left shows the file structure: PROXYPATTERNEXAMPLE, Image.java, ProxyImage.java, ProxyPatternTest.java, and RealImage.java. The main editor window displays the code for ProxyImage.java, which implements the Image interface. The code includes a private RealImage object, a filename, a constructor, and a display() method that uses lazy initialization to load the image from a remote server. The bottom panel shows the terminal output, which includes the command to run the application and the resulting output: 'Loading image from remote server: sample.jpg' and 'Displaying image: sample.jpg'.

```
1 public class ProxyImage implements Image {
2
3     private RealImage realImage;
4     private String fileName;
5
6     public ProxyImage(String fileName) {
7         this.fileName = fileName;
8     }
9
10    @Override
11    public void display() {
12
13        // Lazy Initialization
14        if (realImage == null) {
15            realImage = new RealImage(fileName);
16        }
17
18        // Cached object reused
19        realImage.display();
20    }
21 }
```

PS D:\Cognizant Hands-On\ProxyPatternExample> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vivek\AppData\Roaming\Code\User\workspaceStorage\63ba311c94dbf57d402fabec8e1f22e\redhat.java\jdt_ws\ProxyPatternExample_a54fad7\bin' 'ProxyPatternTest'

Loading image from remote server: sample.jpg
Displaying image: sample.jpg

Second Display:
Displaying image: sample.jpg

The screenshot shows the Visual Studio Code editor with the 'ProxyPatternExample' project open. The Explorer sidebar on the left shows the file structure: PROXYPATTERNEXAMPLE, Image.java, ProxyImage.java, ProxyPatternTest.java, and RealImage.java. The main editor window displays the code for ProxyPatternTest.java, which contains a main method that creates a ProxyImage object and calls its display() method twice. The bottom panel shows the terminal output, which includes the command to run the application and the resulting output: 'Loading image from remote server: sample.jpg' and 'Displaying image: sample.jpg'.

```
1 public class ProxyPatternTest {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         Image image = new ProxyImage(fileName: "sample.jpg");
7
8         System.out.println(x: "First Display:");
9         image.display();
10
11        System.out.println(x: "\nSecond Display:");
12        image.display();
13    }
14 }
```

PS D:\Cognizant Hands-On\ProxyPatternExample> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vivek\AppData\Roaming\Code\User\workspaceStorage\63ba311c94dbf57d402fabec8e1f22e\redhat.java\jdt_ws\ProxyPatternExample_a54fad7\bin' 'ProxyPatternTest'

Loading image from remote server: sample.jpg
Displaying image: sample.jpg

Second Display:
Displaying image: sample.jpg

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Run: ProxyPatternTest + - [ ] ... | X
PS D:\Cognizant Hands-On\ProxyPatternExample> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vivek\AppData\Roaming\Code\User\workspaceStorage\63ba311c94dbf57d402fabec8e1f22e\redhat.java\jdt_ws\ProxyPatternExample_a54fadf7\bin' 'ProxyPatternTest'
First Display:
Loading image from remote server: sample.jpg
Displaying image: sample.jpg

Second Display:
Displaying image: sample.jpg
PS D:\Cognizant Hands-On\ProxyPatternExample>
```

Ln 13, Col 2 Spaces: 4 UTF-8 CRLF () Java [] []